AI ASSISTED CODING

ASSIGNMENT-21.3

Name: T. Swetha

Htno:2303A51317

Batch:05

Task 1 – AI-Assisted Pair Programming
Example:
Use AI to generate an initial Expense Tracker Application.
• Ask AI to create a basic program to record daily expenses and categories.
• Prompt AI to suggest improvements such as:
o Monthly expense summary
o Input validation
o Code refactoring into functions
• Compare the initial and improved versions to understand AI's role in collaborative coding.

Prompt:

Create a simple **Python Expense Tracker** program.
First give a **basic version** to add daily expenses and categories.
Then improve the code by adding:

- input validation
- functions
- better structure

Also compare the first code and improved code, and explain how AI helped in writing and improving the program.

E Create a simple Python Expense Tracker program. First give a basic version to add daily expenses and categories. Then improve the code by adding: input validation functions better structure Also compare the first code and improved code, and explain how AI helped in writing and improving the program.

Code:

```python
def get_expense_input():
    """Prompts the user for expense amount and category with validation."""
    while True:
        amount_input = input("Enter expense amount (or 'done' to finish): ")
        if amount_input.lower() == 'done':
            return None, None # Signal to stop input

        try:
            amount = float(amount_input)
            if amount <= 0:
                print("Expense amount must be positive. Please try again.")
                continue
        except ValueError:
            print("Invalid amount. Please enter a number.")
            continue

        category = input("Enter category for this expense: ").strip()
        if not category:
            print("Category cannot be empty. Please try again.")
            continue

        return amount, category

def add_expense(expenses_list, amount, category):
    """Adds a new expense to the list."""
    expenses_list.append({'amount': amount, 'category': category})
    print(f"Added: ${amount:.2f} to {category}")

def display_expenses(expenses_list):
    """Displays all recorded expenses with a summary."""
    print("\n--- Your Expenses ---")
    if not expenses_list:
        print("No expenses recorded.")
    else:
        total_expenses = 0
        print("Category         Amount")
        print("----------------------")
        for exp in expenses_list:
            print(f"{exp['category']:<15} ${exp['amount']:.2f}")
            total_expenses += exp['amount']
        print("----------------------")
        print(f"Total:           ${total_expenses:.2f}")

def main():
    """Main function to run the improved expense tracker."""
    expenses = []
    print("--- Improved Expense Tracker ---")

    while True:
        amount, category = get_expense_input()
        if amount is None:
            break
        add_expense(expenses, amount, category)

    display_expenses(expenses)

main()
```

Output:

```
--- Improved Expense Tracker ---
Enter expense amount (or 'done' to finish): 12
Enter category for this expense: food
Added: $12.00 to food
Enter expense amount (or 'done' to finish): done

--- Your Expenses ---
Category          Amount
----------------------
food             $12.00
----------------------
Total:            $12.00
```

Explanation:

- In this task, AI is used as a **coding helper**.
  First, AI creates a simple **Expense Tracker** program to store daily expenses and categories.
- Then, AI improves the code by adding **input checking**, **functions**, and better structure.
- By comparing both versions, we can understand how AI helps in writing, improving, and organizing code.
  This shows the role of AI in **pair programming** and collaborative coding.

Task 2 – Version Control Integration

Example:

Manage a Student Grade Calculator project using Git.

• Initialize a Git repository for the project.

• Create a new branch to add AI-assisted features such as:

o Grade classification (A, B, C, etc.)

o Average and percentage calculation

• Commit changes with meaningful commit messages.

• Merge the feature branch back into the main branch and resolve conflicts if any.

Prompt:

In this task, AI is used as a **coding helper**. First, AI creates a simple **Expense Tracker** program to store daily expenses and categories. Then, AI improves the code by adding **input checking**, **functions**, and better structure. By comparing both versions, we can understand how AI helps in writing, improving, and organizing code. This shows the role of AI in **pair programming** and collaborative coding.

E In this task, AI is used as a coding helper. First, AI creates a simple Expense Tracker program to store daily expenses and categories. Then, AI improves the code by adding input checking, functions, and better structure. By comparing both versions, we can understand how AI helps in writing, improving, and organizing code. This shows the role of AI in pair programming and collaborative coding.

Code:

```python
def get_expense_input():
    """Prompts the user for expense amount and category with validation."""
    while True:
        amount_input = input("Enter expense amount (or 'done' to finish): ")
        if amount_input.lower() == 'done':
            return None, None # Signal to stop input

        try:
            amount = float(amount_input)
            if amount <= 0:
                print("Expense amount must be positive. Please try again.")
                continue
        except ValueError:
            print("Invalid amount. Please enter a number.")
            continue

        category = input("Enter category for this expense: ").strip()
        if not category:
            print("Category cannot be empty. Please try again.")
            continue

        return amount, category

def add_expense(expenses_list, amount, category):
    """Adds a new expense to the list."""
    expenses_list.append({'amount': amount, 'category': category})
    print(f"Added: ${amount:.2f} to {category}")

def display_expenses(expenses_list):
    """Displays all recorded expenses with a summary."""
    print("\n--- Your Expenses ---")
    if not expenses_list:
        print("No expenses recorded.")
    else:
        total_expenses = 0
        print("Category         Amount")
        print("----------------------")
        for exp in expenses_list:
            print(f"{exp['category']:<15} ${exp['amount']:.2f}")
            total_expenses += exp['amount']
        print("----------------------")
        print(f"Total:           ${total_expenses:.2f}")

def calculate_category_totals(expenses_list):
    """Calculates the total spending for each category."""
    category_totals = {}
    for exp in expenses_list:
        category = exp['category']
        amount = exp['amount']
        category_totals[category] = category_totals.get(category, 0) + amount
    return category_totals

def main():
    """Main function to run the improved expense tracker."""
    expenses = []
    print("--- Improved Expense Tracker ---")

    while True:
        amount, category = get_expense_input()
        if amount is None:
```

```python
    while True:
        amount, category = get_expense_input()
        if amount is None:
            break
        add_expense(expenses, amount, category)

    display_expenses(expenses)

    # Display category totals
    print("\n--- Spending Summary by Category ---")
    if expenses:
        category_summary = calculate_category_totals(expenses)
        if category_summary:
            for category, total in category_summary.items():
                print(f"{category:<15} ${total:.2f}")
        else:
            print("No expenses recorded to summarize by category.")
    else:
        print("No expenses recorded to summarize by category.")

# Run the main program
main()
```

Output:

```
--- Improved Expense Tracker ---
Enter expense amount (or 'done' to finish): 30
Enter category for this expense: Cloth
Added: $30.00 to Cloth
Enter expense amount (or 'done' to finish): finich
Invalid amount. Please enter a number.
Enter expense amount (or 'done' to finish): finish
Invalid amount. Please enter a number.
Enter expense amount (or 'done' to finish): 300
Enter category for this expense: food
Added: $300.00 to food
Enter expense amount (or 'done' to finish): finish
Invalid amount. Please enter a number.
Enter expense amount (or 'done' to finish): 30000
Enter category for this expense: cloth
Added: $30000.00 to cloth
Enter expense amount (or 'done' to finish): Done

--- Your Expenses ---
Category         Amount
---------------------
Cloth           $30.00
food            $300.00
cloth           $30000.00
---------------------
Total:           $30330.00

--- Spending Summary by Category ---
Cloth           $30.00
food            $300.00
cloth           $30000.00
```

Explanation:

- Git was used to track changes in the project and manage different versions of the code efficiently.
- A separate branch was created to add AI-assisted features without affecting the main codebase.
- New features such as grade classification and percentage calculation were added using AI suggestions.
- Changes were committed with meaningful messages, making the development process easy to track.
- The feature branch was merged back into the main branch, demonstrating collaborative development and version control.

Task 3 – AI for Commit Message Generation
Example:
Use AI to generate commit messages for a Password Validation
Module.
• Modify the code to add rules like:
o Minimum length
o Special character requirement
• Ask AI to generate commit messages describing these updates.
• Compare AI-generated messages with human-written ones and analyze clarity.

Prompt:

Create a simple **Python Password Validation Module**.
First give a basic version.
Then improve it by adding:

- minimum password length
- special character requirement

After updating the code, generate suitable **Git commit messages** for these changes.
Also compare **AI-generated commit messages** with **human-written commit messages**

E Create a simple Python Password Validation Module. First give a basic version. Then improve it by adding: minimum password length special character requirement After updating the code, generate suitable Git commit messages for these changes. Also compare AI-generated commit messages with human-written commit messages

Code:

```python
import re

def validate_password(password, min_length=8, require_special_char=True):
    """
    Validates a password for minimum length and special characters.

    Args:
        password (str): The password string to validate.
        min_length (int): The minimum required length for the password.
        require_special_char (bool): Whether a special character is required.

    Returns:
        bool: True if the password is valid, False otherwise.
        list: A list of error messages if validation fails.
    """
    errors = []

    if len(password) < min_length:
        errors.append(f"Password must be at least {min_length} characters long.")

    if require_special_char:
        # Check for at least one special character (non-alphanumeric and non-underscore)
        if not re.search(r'[!@#$%^&*()_+\-=\[\]{};\'\":|,.<>/?`~]', password):
            errors.append("Password must contain at least one special character.")

    if errors:
        return False, errors
    else:
        return True, []

# --- Demonstration ---
print("--- Password Validation Demo ---")

# Test cases
password_1 = "MySecureP@ss1"
password_2 = "short"
password_3 = "NoSpecialChars123"
password_4 = "AnotherGoodP@ss"

# Test with valid password
is_valid, msgs = validate_password(password_1)
print(f"\nPassword: '{password_1}' - Valid: {is_valid}")
if not is_valid:
    for msg in msgs:
        print(f" - {msg}")

# Test with short password
is_valid, msgs = validate_password(password_2)
print(f"\nPassword: '{password_2}' - Valid: {is_valid}")
if not is_valid:
    for msg in msgs:
        print(f" - {msg}")

# Test with no special characters (when required)
is_valid, msgs = validate_password(password_3)
print(f"\nPassword: '{password_3}' - Valid: {is_valid}")
if not is_valid:
    for msg in msgs:
        print(f" - {msg}")
```

Output:

```
--- Password Validation Demo ---

Password: 'MySecureP@ss1' - Valid: True

Password: 'short' - Valid: False
 - Password must be at least 8 characters long.
 - Password must contain at least one special character.

Password: 'NoSpecialChars123' - Valid: False
 - Password must contain at least one special character.

Password: 'AnotherGoodP@ss' (min_length=10) - Valid: True
```

Explanation:

- In this task, AI is used to help write **commit messages** for code changes.
  First, a basic **Password Validation Module** is created.
- Then, new rules like **minimum length** and **special character checking** are added.
  After that, AI generates commit messages to describe these updates.
- By comparing AI-generated messages with human-written messages, we can understand how clearly AI explains code changes.

Task 4 – Pair Programming Simulation
Scenario: Collaboratively develop a Quiz Management System.
• Student A creates the initial quiz program with questions and answers.
• Student B, with AI assistance, adds features such as:

o Score calculation
o Input validation
• Both students collaborate using Git branching and merging to integrate their changes.

<u>Prompt:</u>

Create a simple **Python Quiz Management System**.
First give a **basic quiz program** with questions and answers.
Then improve it by adding:

- score calculation
- input validation

Also explain how two students can work on this project using **pair programming**, **Git branching**, and **merging** to combine their changes.

E Create a simple Python Quiz Management System. First give a basic quiz program with questions and answers. Then improve it by adding: score calculation input validation Also explain how two students can work on this project using pair programming, Git branching, and merging to combine their changes.

<u>Code:</u>

```python
# Basic Quiz Program

questions = [
    {
        "prompt": "What is the capital of France?\nA. Berlin\nB. Madrid\nC. Paris\nD. Rome\n",
        "answer": "C"
    },
    {
        "prompt": "What is 2 + 2?\nA. 3\nB. 4\nC. 5\nD. 6\n",
        "answer": "B"
    },
    {
        "prompt": "Which planet is known as the Red Planet?\nA. Earth\nB. Mars\nC. Jupiter\nD. Venus\n",
        "answer": "B"
    }
]

print("--- Welcome to the Basic Quiz! ---")

def run_quiz(questions):
    for question in questions:
        print("\n" + question["prompt"])
        user_answer = input("Your answer (A, B, C, or D): ").upper()
        if user_answer == question["answer"]:
            print("Correct!")
        else:
            print(f"Incorrect. The correct answer was {question['answer']}.")

run_quiz(questions)

print("\n--- Quiz End ---")
```

Output:

```
•••  --- Welcome to the Basic Quiz! ---

     What is the capital of France?
     A. Berlin
     B. Madrid
     C. Paris
     D. Rome

     Your answer (A, B, C, or D): A
     Incorrect. The correct answer was C.

     What is 2 + 2?
     A. 3
     B. 4
     C. 5
     D. 6

     Your answer (A, B, C, or D): B
     Correct!

     Which planet is known as the Red Planet?
     A. Earth
     B. Mars
     C. Jupiter
     D. Venus

     Your answer (A, B, C, or D): C
     Incorrect. The correct answer was B.

     --- Quiz End ---
```

Explanation:

- In this task, two students work together to build a **Quiz Management System**. Student A creates the basic quiz program with questions and answers.
- Student B uses AI to improve the program by adding **score calculation** and **input validation**.
- Then both students use **Git branching and merging** to combine their work into one final program.
- This task shows how AI can support **pair programming** and teamwork in coding.